

Symmetric Ciphers

IDEA, AES

IDEA

- The International Data Encryption Algorithm (IDEA) is a symmetric-key, block cipher.
- It was published in 1991 by Lai, Massey, and Murphy .
- IDEA is a modification of the Proposed Encryption Standard (PES) that was published in 1990 by Lai and Massy.
- PES was designed as a replacement for the Data Encryption Standard (DES).

IDEA

- The algorithm was modified and published in 1991 after Biham and Shamir described the technique of differential cryptanalysis.
- The new algorithm was called the Improved Proposed Encryption Standard (IPES).
- Its name changed to IDEA in 1992.
- Today, IDEA is obsolete, but its algorithm can be a useful teaching tool to help students bridge the gap between DES, and AES.
- DES uses XOR but no algebraic operations.

IDEA

- IDEA operates with 64-bit blocks with 128-bit key
- The 64 bit plain text is divided into 4 sub-blocks, each of 16 bits in size
- The 128 bit key is used to generate 52 sub keys of length 16 bits.
- Number of identical rounds is 8 and in each round 6 keys are used.
- Like this 48 keys are used and at last round (also known as half-round) another 4 keys

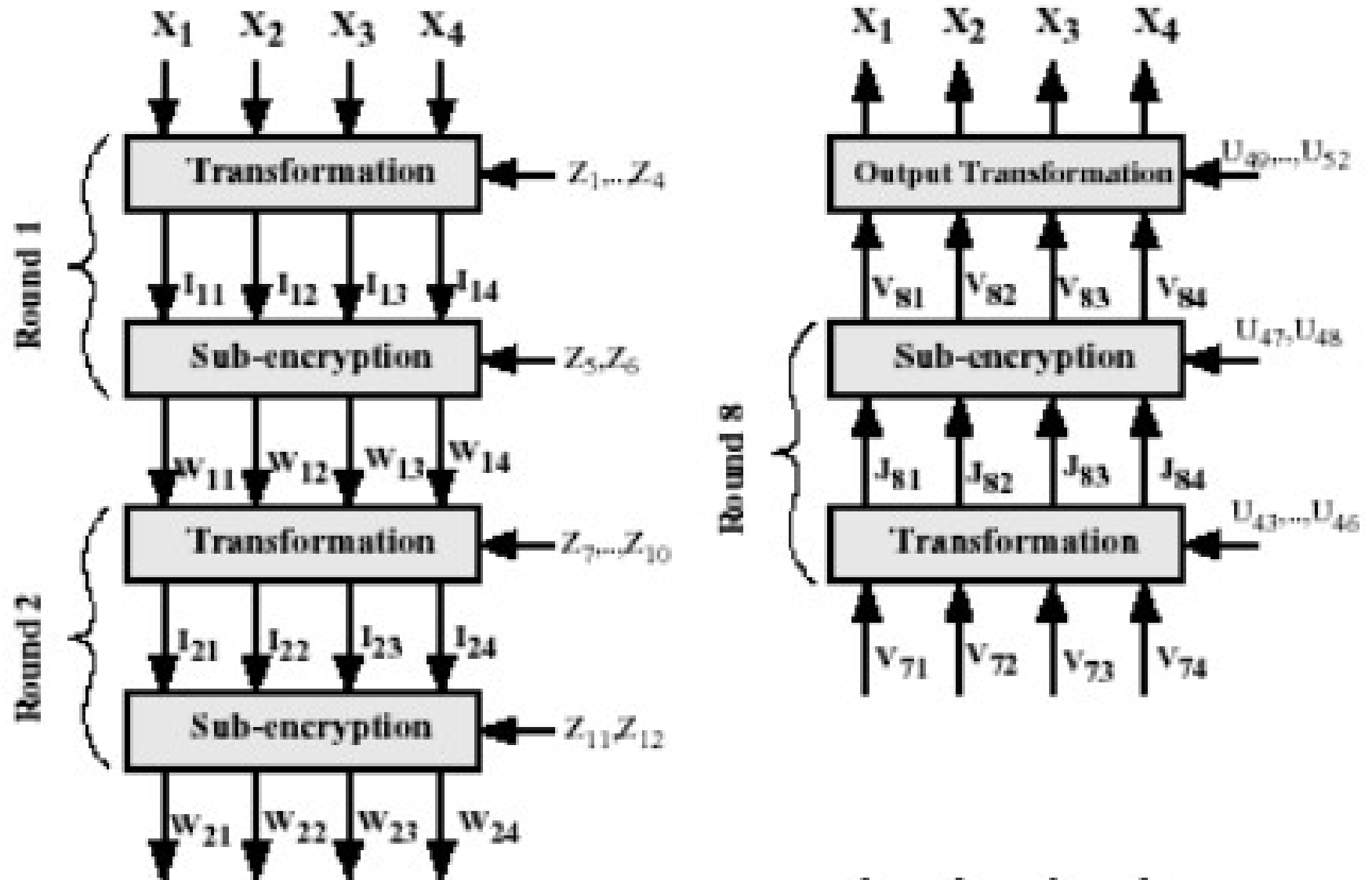
IDEA

- IDEA derives much of its security by mixing polynomial modular operations, Bitwise XOR
- The algorithm structure has been chosen such that when different key sub-blocks are used, the encryption process is identical to the decryption process

IDEA: Algorithm

- 52 keys of 16bit are generated from 128 bit initial key.
- Plaintext is divided into blocks of 64 bits.
- 64bit block is divided into 4 sub-blocks of 16bit.
- Round operation is performed for 8 rounds on sub blocks.
- Output transformation is performed to get the ciphertext.

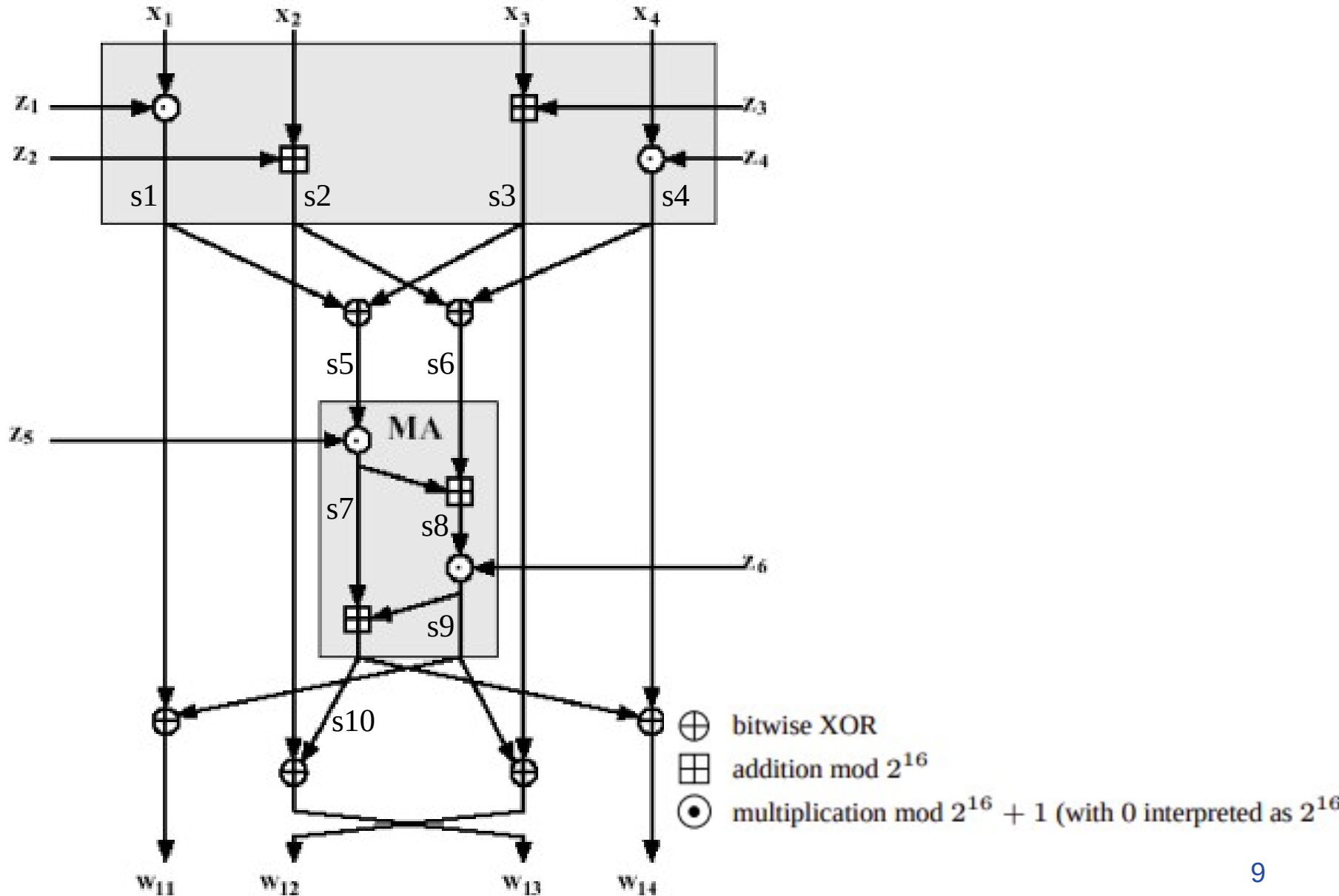
IDEA:



IDEA: Subkey generation

- Altogether 52 16-bit subkeys are generated from 128-bit encryption key.
- The first eight subkeys, labeled Z1, Z2, Z3, Z4, Z5, Z6, Z7, Z8 are taken directly from the key.
- Z1 being equal to the first 16 bits, Z2 corresponding to the next 16 bits, and so on.
- Then a circular left shift of 25 bit positions is applied to the key, and the next eight subkeys are extracted.
- This procedure is repeated until all 52 subkeys are generated.

IDEA: Round Operation



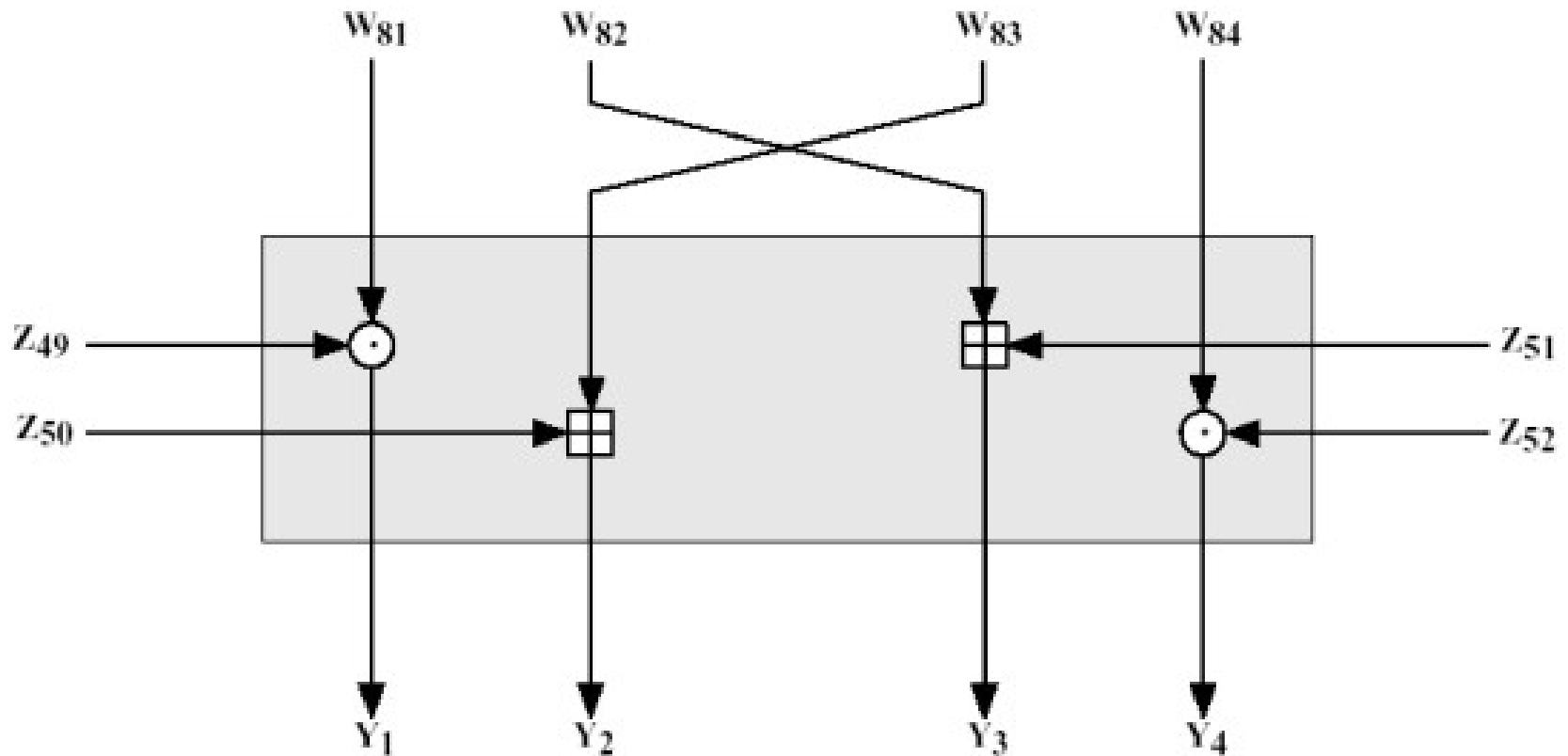
IDEA: round operation

- $s1 = x1 \times z1 \bmod 2^{16} + 1$
- $s2 = x2 + z2 \bmod 2^{16}$
- $s3 = x3 + z3 \bmod 2^{16}$
- $s4 = x4 \times z4 \bmod 2^{16} + 1$
- $s5 = s1 \text{ xor } s3$
- $s6 = s2 \text{ xor } s4$
- $s7 = s5 \times z5$
- $s8 = s6 + s7$

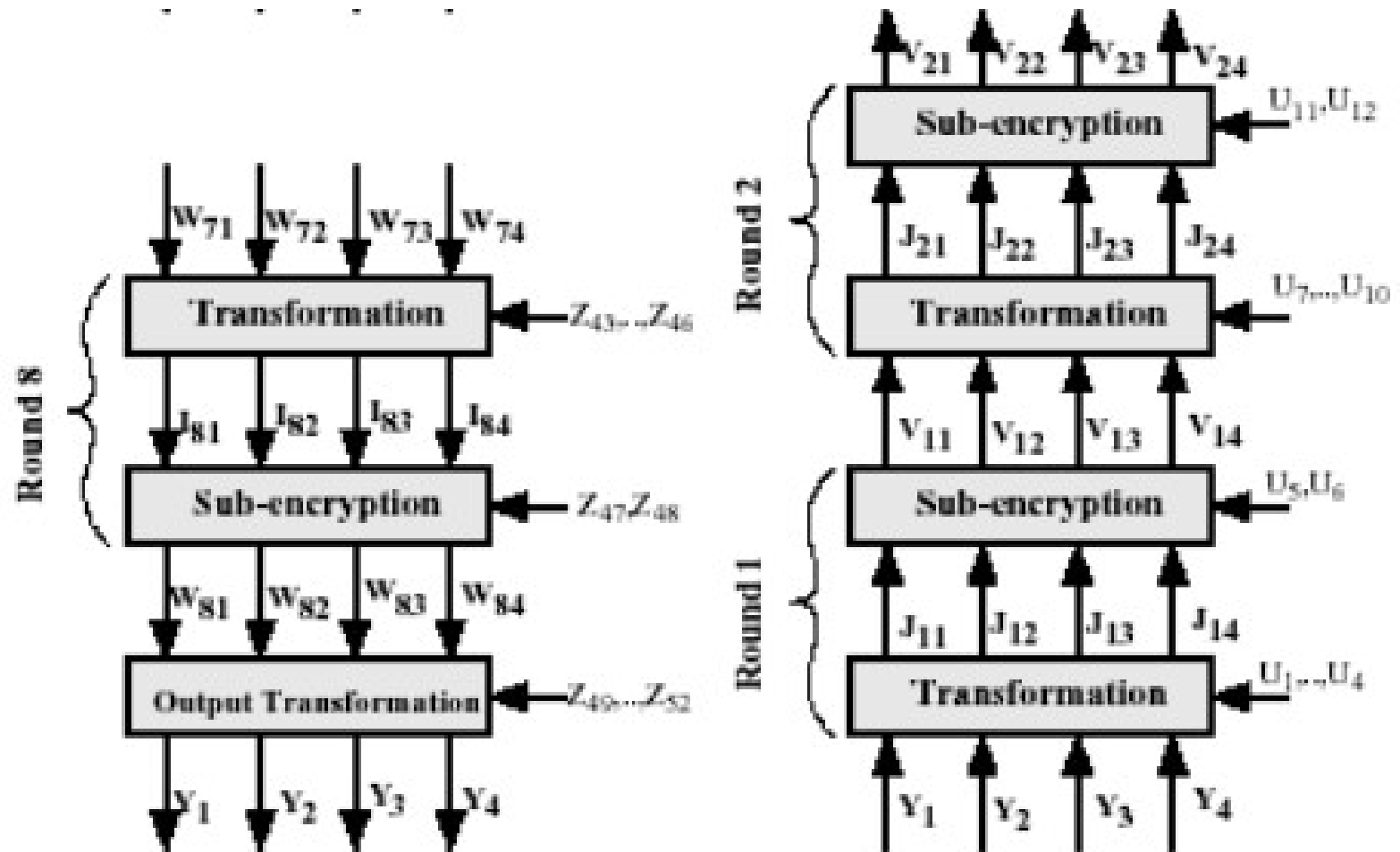
IDEA: round operation

- $s_9 = s_8 \times z_6 \bmod 2^{16} + 1$
- $s_{10} = s_9 + s_7 \bmod 2^{16}$
- $w_{11} = s_1 \text{ xor } s_9$
- $w_{12} = s_3 \text{ xor } s_9$
- $w_{13} = s_2 \text{ xor } s_{10}$
- $w_{14} = s_4 \text{ xor } s_{10}$

IDEA: Output Transformation



IDEA:



IDEA: Simplified

- Simple version of IDEA is using 16 bit plaintext and 32 bit key to generate 16 bit ciphertext.
- The plaintext is divided into 4 blocks of 4bits.
- The addition is modulo 2^{16}
- Multiplication is modulo $2^4 + 1$
- All operations are same to original version.
- For key generation left circular shift by 6 bits is performed.
- Total 28 subkeys are generated for 5 rounds

IDEA: Simplified Example

- Let us take a 32-bit key
1101 1100 0110 1111 0011 1111 0101 1001
- Left shift by 6 bits
- 0001 1011 1100 1111 1101 0110 0111 0111
- 1111 0011 1111 0101 1001 1101 1100 0110
- 1111 1101 0110 0111 0111 0001 1011 1100
- Now, 28 keys are

The 32-bit key is 1101 1100 0110 1111 0011 1111 0101 1001.

	Z_1	Z_2	Z_3	Z_4	Z_5	Z_6
Round 1	1101	1100	0110	1111	0011	1111
Round 2	0101	1001★	0001	1011	1100	1111
Round 3	1101	0110	0111	0111★	1111	0011
Round 4	1111	0101	1001	1101	1100	0110★
Round 5	1111	1101	0110	0111		

IDEA: round operation

- Let the plaintext be 1001110010101100
- The 4 blocks of plaintext are
 $x_1=1001$ $x_2=1100$ $x_3=1010$ $x_4=1100$
- $s_1 = x_1 \times z_1 = 1001 \times 1101 = 1111$
- $s_2 = x_2 + z_2 = 1100 + 1100 = 1000$
- $s_3 = x_3 + z_3 = 1010 + 0110 = 0000$
- $s_4 = x_4 \times z_4 = 1100 \times 1111 = 1010$
- $s_5 = s_1 \text{ xor } s_3 = 1111 \text{ xor } 0000 = 1111$
- $s_6 = s_2 \text{ xor } s_4 = 1000 \text{ xor } 1010 = 0010$

IDEA: round operation

- $s_7 = s_5 \times z_5 = 1111 \times 0011 = 1011$
- $s_8 = s_6 + s_7 = 0010 + 1011 = 1101$
- $s_9 = s_8 \times z_6 = 1101 \times 1111 = 1000$
- $s_{10} = s_9 + s_7 = 1000 + 1011 = 0011$
- $w_{11} = s_1 \text{ xor } s_9 = 1111 \text{ xor } 1000 = 0111$
- $w_{12} = s_3 \text{ xor } s_9 = 0000 \text{ xor } 1000 = 1000$
- $w_{13} = s_2 \text{ xor } s_{10} = 1000 \text{ xor } 0011 = 1011$
- $w_{14} = s_4 \text{ xor } s_{10} = 1010 \text{ xor } 0011 = 1001$

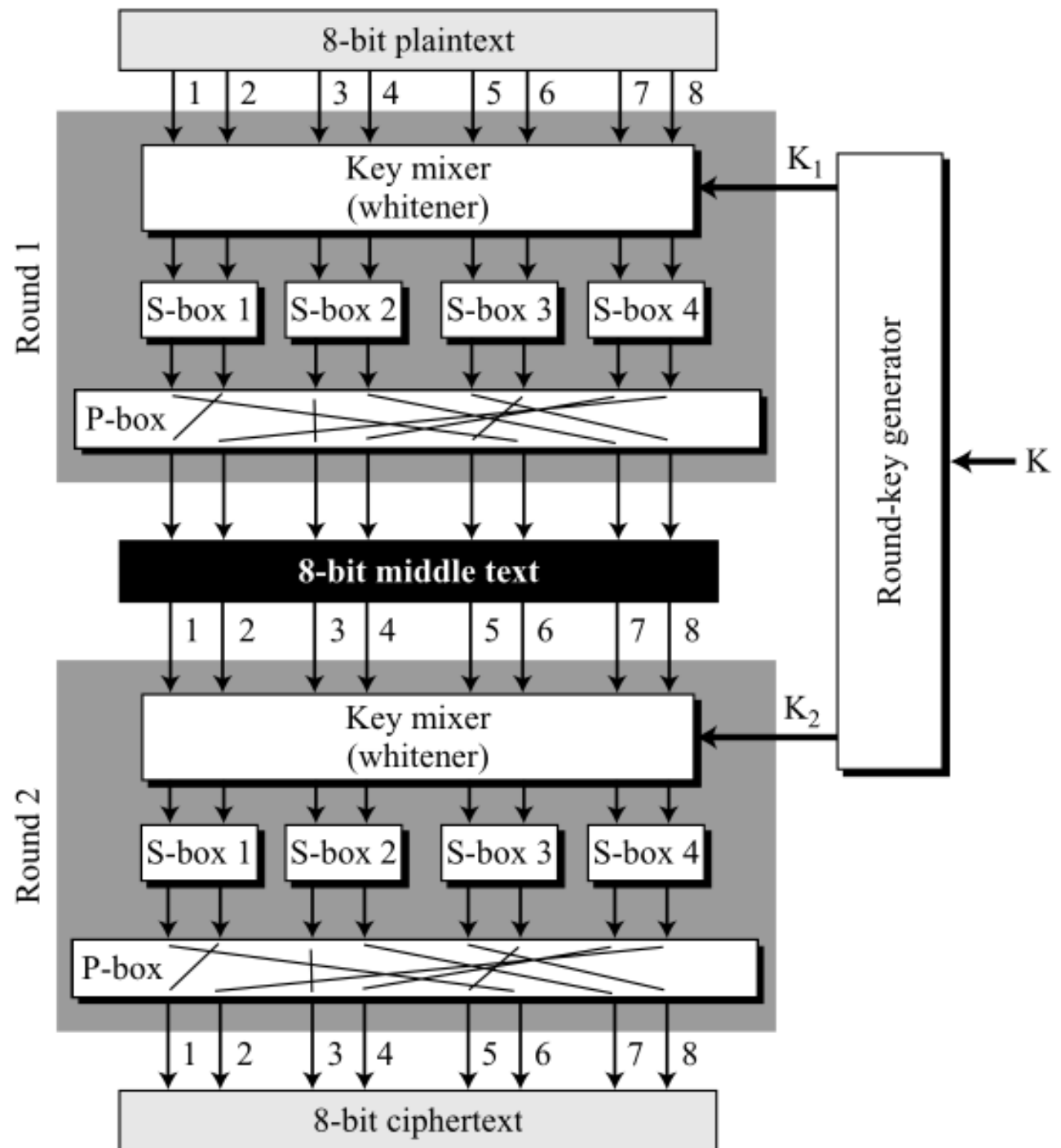
Substitution Permutation Network

- A non-Feistel cipher uses only invertible components.
- It is also known as SPN.
- A component in the encryption cipher has the corresponding component in the decryption cipher.
- For example, S-boxes need to have an equal number of inputs and outputs to be compatible.
- No compression or expansion P-boxes are allowed, because they are not invertible.

SPN

- In SPN, there is no need to divide the plaintext into two halves as we saw in the Feistel ciphers.
- AES is an example of SPN.
- Figure below is a SPN

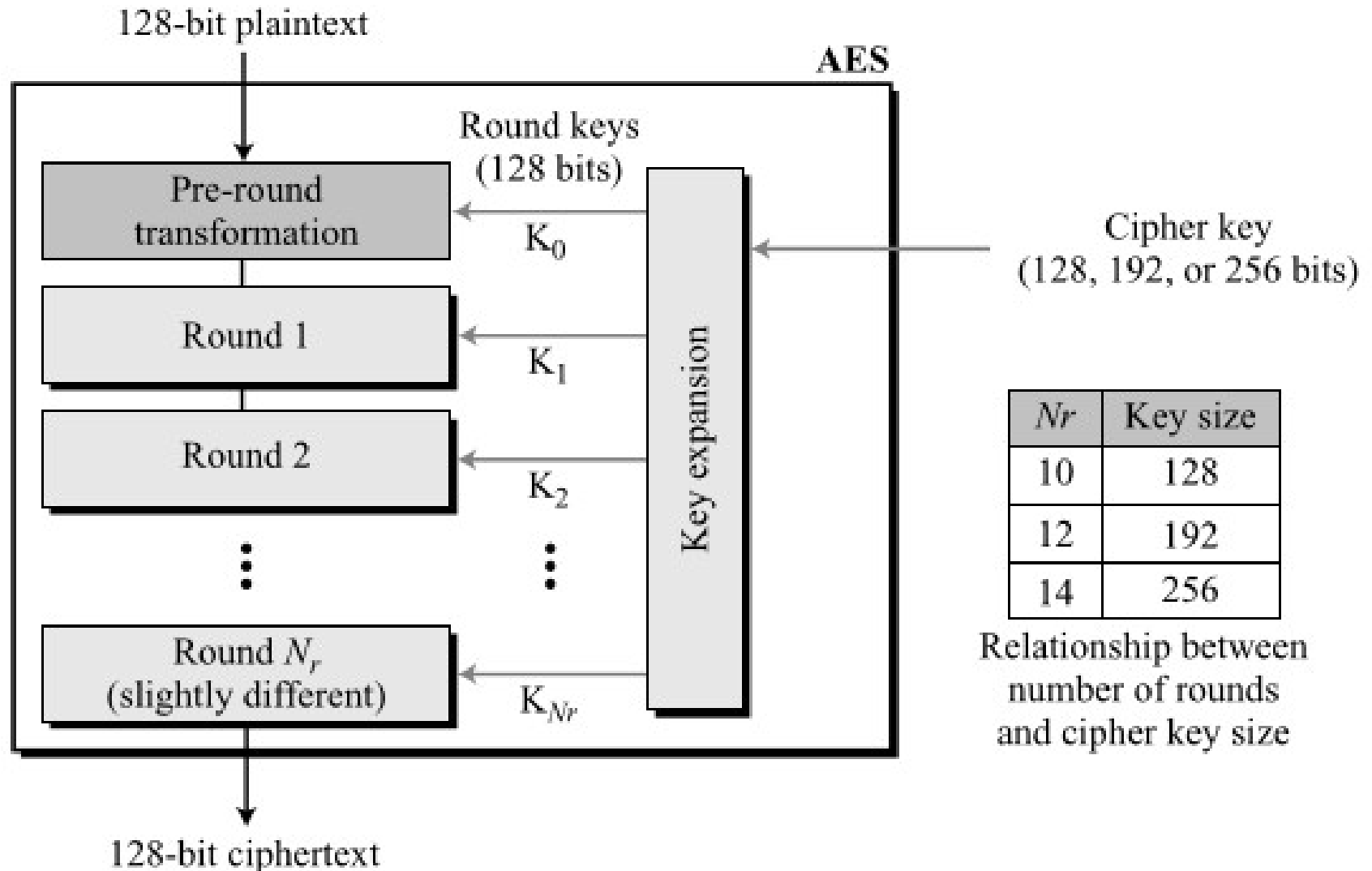
SPN



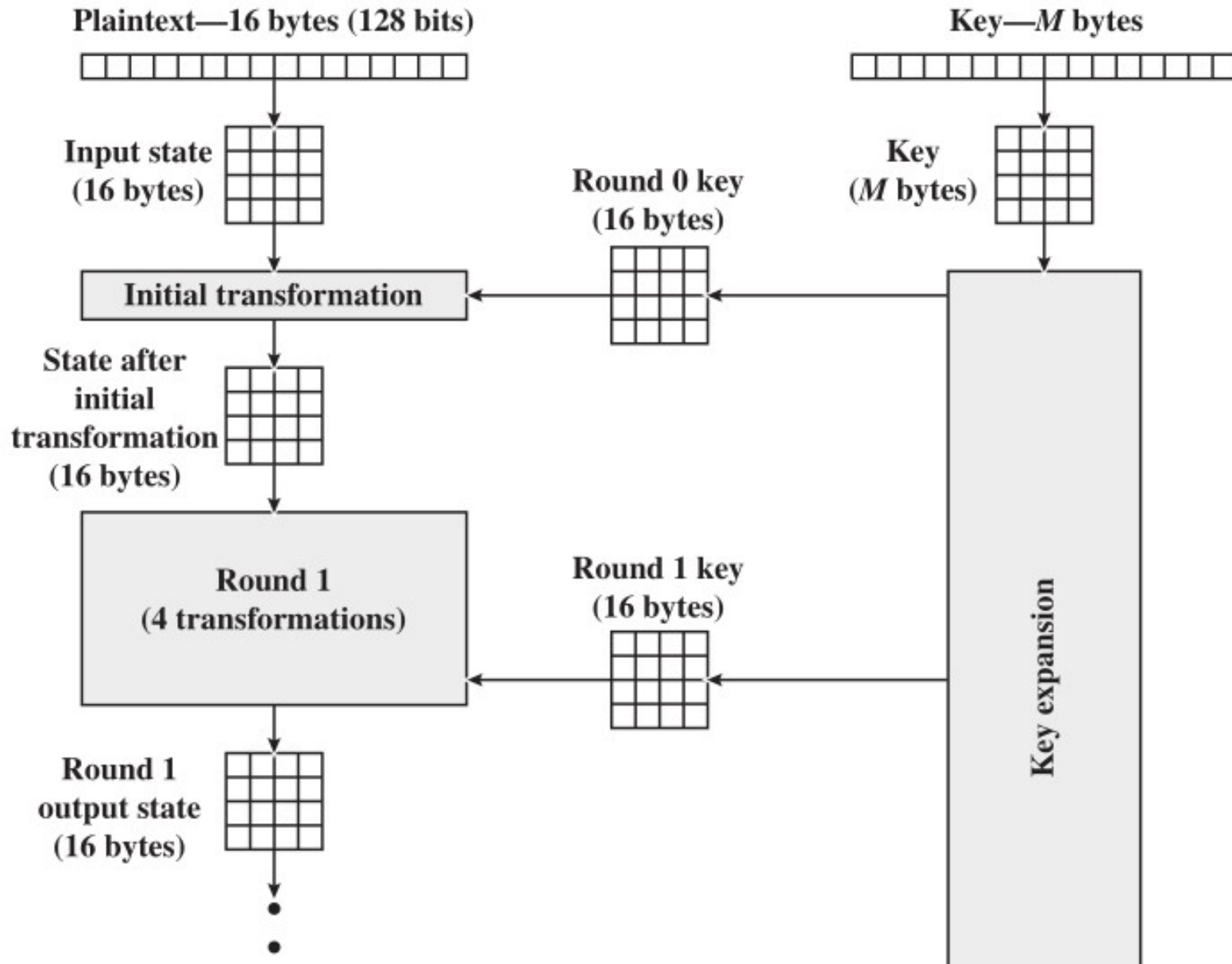
Advanced Encryption Standard

- AES was published by NIST in Dec, 2001.
- AES is a non-Feistel cipher that encrypts and decrypts a data block of 128 bits.
- It can use 10, 12, or 14 rounds with respective key sizes 128, 192, or 256 bits.
- Figure below shows the general design for the encryption algorithm
- The decryption algorithm (called inverse cipher) is similar,
- the round keys are applied in the reverse order

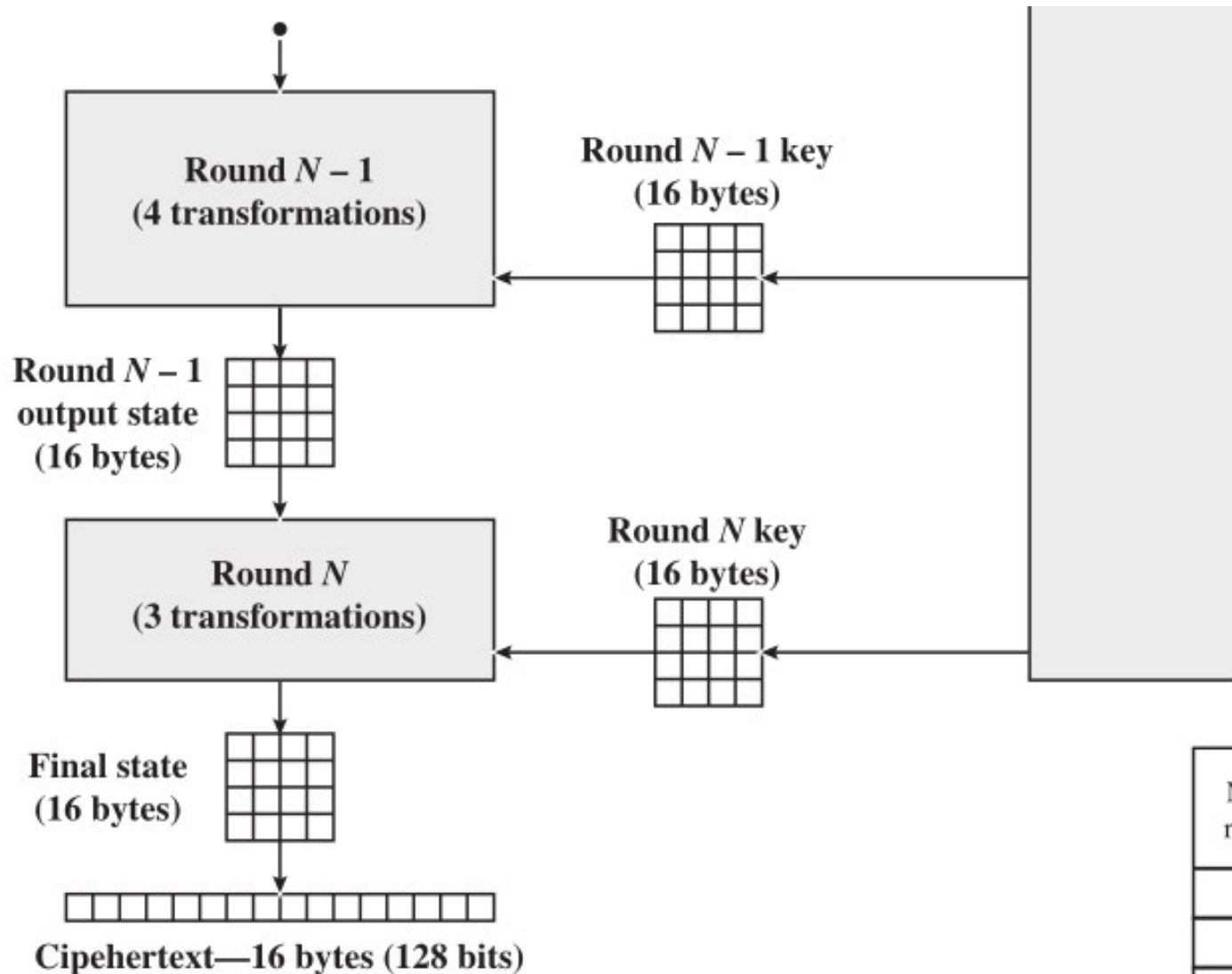
Advanced Encryption Standard



Advanced Encryption Standard



Advanced Encryption Standard



No. of rounds	Key Length (bytes)
10	16
12	24
14	32

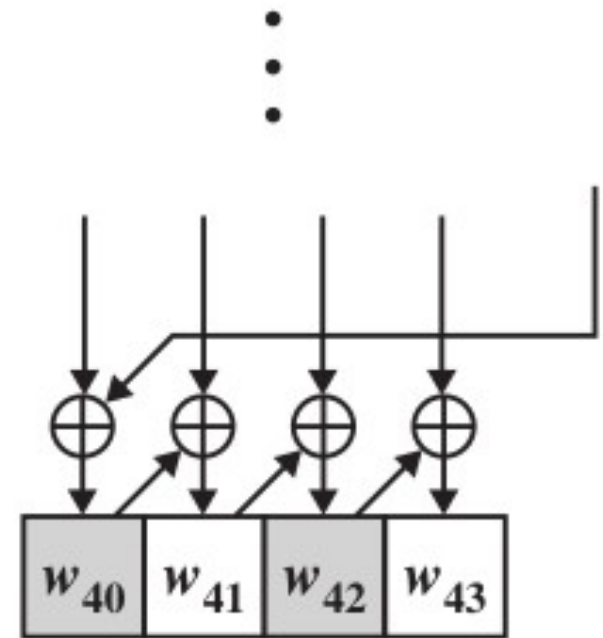
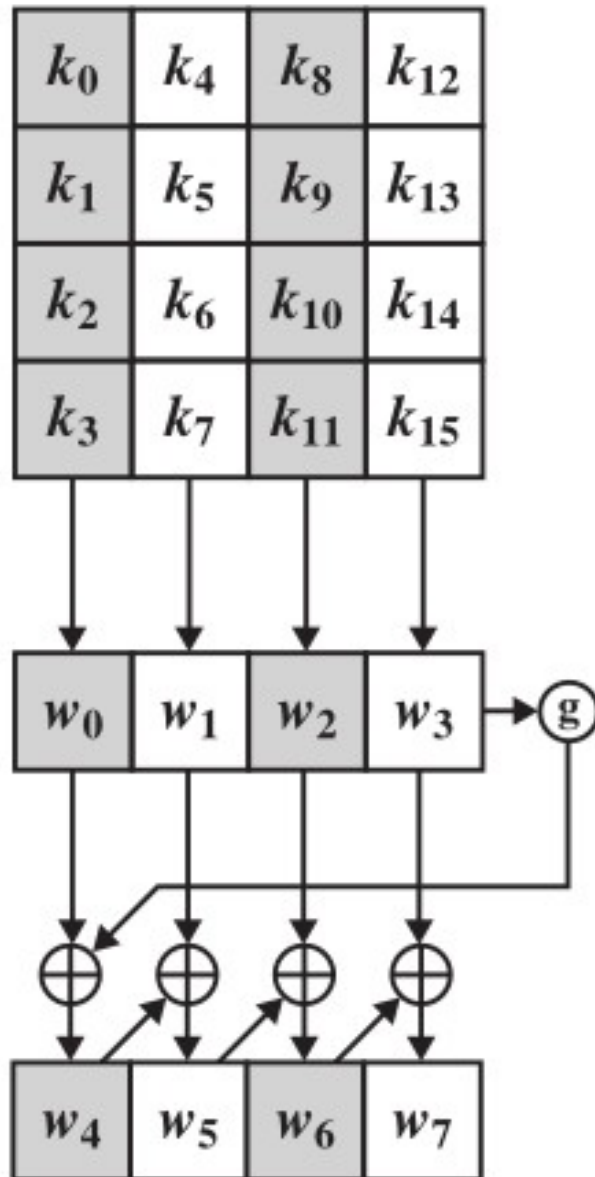
Advanced Encryption Standard

- 1. Generate round keys using key expansion algorithm
- 2. Perform initial transformation of plaintext using K_0
- 3. Perform round transformation
- The output of final round is ciphertext.
- It maintains state matrix to store different intermediate results.
- Initially state matrix consists of plaintext.
- State matrix consists of 16 bytes.

Key Expansion

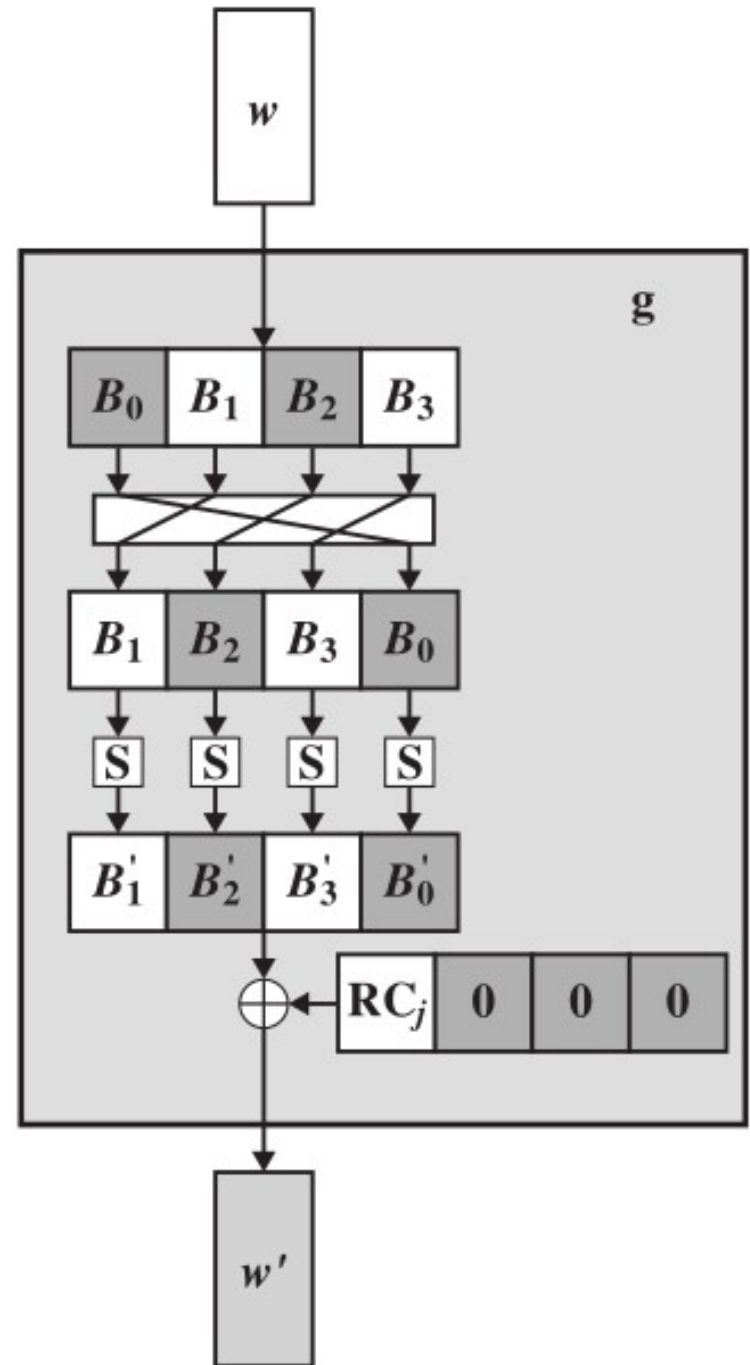
- Key expansion algorithm generates round keys from K_1 to K_{10} if $n=10$.
- It generates these keys by transforming the given key, K_0 .
- Each round keys are 128 bits or four words.
- $K_0 = (k_0, k_1, k_2, k_3, k_4, k_5, k_6, k_7, k_8, k_9, k_{10}, k_{11}, k_{12}, k_{13}, k_{14}, k_{15})$
- Each k_i is two hexadecimal number which is 8bit
- $K_0 = (w_0, w_1, w_2, w_3)$
- Where, $w_0 = (k_0, k_1, k_2, k_3)$, $w_1 = (k_4, k_5, k_6, k_7)$
- $w_2 = (k_8, k_9, k_{10}, k_{11})$, $w_3 = (k_{12}, k_{13}, k_{14}, k_{15})$

Key Expansion



Key Expansion: g function

- Last word is circular left shifted
- Each byte of the word is substituted with the bytes in S-box given below.
- The location of byte is found with the byte in word
- The first nibble in each byte represents row (x) of S-box and second nibble represents column (y)



Key Expansion: g function

Table 5.2 AES S-Boxes

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Key Expansion: g

- Let given key be
- $K_0 = 0f1571c947d9e8590cb7add6af7f6798$

Key Words	Auxiliary Function
$w_0 = 0f \ 15 \ 71 \ c9$ $w_1 = 47 \ d9 \ e8 \ 59$ $w_2 = 0c \ b7 \ ad \ d6$ $w_3 = af \ 7f \ 67 \ 98$	$RotWord(w_3) = 7f \ 67 \ 98 \ af = x_1$ $SubWord(x_1) = d2 \ 85 \ 46 \ 79 = y_1$ $Rcon(1) = 01 \ 00 \ 00 \ 00$ $y_1 \oplus Rcon(1) = d3 \ 85 \ 46 \ 79 = z_1$
$w_4 = w_0 \oplus z_1 = dc \ 90 \ 37 \ b0$ $w_5 = w_4 \oplus w_1 = 9b \ 49 \ df \ e9$ $w_6 = w_5 \oplus w_2 = 97 \ fe \ 72 \ 3f$ $w_7 = w_6 \oplus w_3 = 38 \ 81 \ 15 \ a7$	$RotWord(w_7) = 81 \ 15 \ a7 \ 38 = x_2$ $SubWord(x_2) = 0c \ 59 \ 5c \ 07 = y_2$ $Rcon(2) = 02 \ 00 \ 00 \ 00$ $y_2 \oplus Rcon(2) = 0e \ 59 \ 5c \ 07 = z_2$

- Rotword, x_1 is last word (w_3) of K_0
- Subword, y_1 is substituted word of x_1 from s-box
- Rcon is round constant word

Key Expansion: g function

- Different round constant words are given below.

Table 7.4 *RCon constants*

<i>Round</i>	<i>Constant (RCon)</i>	<i>Round</i>	<i>Constant (RCon)</i>
1	(<u>01</u> 00 00 00) ₁₆	6	(<u>20</u> 00 00 00) ₁₆
2	(<u>02</u> 00 00 00) ₁₆	7	(<u>40</u> 00 00 00) ₁₆
3	(<u>04</u> 00 00 00) ₁₆	8	(<u>80</u> 00 00 00) ₁₆
4	(<u>08</u> 00 00 00) ₁₆	9	(<u>1B</u> 00 00 00) ₁₆
5	(<u>10</u> 00 00 00) ₁₆	10	(<u>36</u> 00 00 00) ₁₆

Key Expansion

- To generate the round keys, the first word of previous round is xored with output of g function.
- This give first word of the round key.
- For second word, the second word of previous round key and first word of the current key is xored.
- In this way, all round keys are found.
- The round keys are given below

Key Expansion

Key Words	Auxiliary Function
$w_0 = 0f\ 15\ 71\ c9$ $w_1 = 47\ d9\ e8\ 59$ $w_2 = 0c\ b7\ ad\ d6$ $w_3 = af\ 7f\ 67\ 98$	$RotWord(w_3) = 7f\ 67\ 98\ af = x_1$ $SubWord(x_1) = d2\ 85\ 46\ 79 = y_1$ $Rcon(1) = 01\ 00\ 00\ 00$ $y_1 \oplus Rcon(1) = d3\ 85\ 46\ 79 = z_1$
$w_4 = w_0 \oplus z_1 = dc\ 90\ 37\ b0$ $w_5 = w_4 \oplus w_1 = 9b\ 49\ df\ e9$ $w_6 = w_5 \oplus w_2 = 97\ fe\ 72\ 3f$ $w_7 = w_6 \oplus w_3 = 38\ 81\ 15\ a7$	$RotWord(w_7) = 81\ 15\ a7\ 38 = x_2$ $SubWord(x_2) = 0c\ 59\ 5c\ 07 = y_2$ $Rcon(2) = 02\ 00\ 00\ 00$ $y_2 \oplus Rcon(2) = 0e\ 59\ 5c\ 07 = z_2$
$w_8 = w_4 \oplus z_2 = d2\ c9\ 6b\ b7$ $w_9 = w_8 \oplus w_5 = 49\ 80\ b4\ 5e$ $w_{10} = w_9 \oplus w_6 = de\ 7e\ c6\ 61$ $w_{11} = w_{10} \oplus w_7 = e6\ ff\ d3\ c6$	$RotWord(w_{11}) = ff\ d3\ c6\ e6 = x_3$ $SubWord(x_3) = 16\ 66\ b4\ 83 = y_3$ $Rcon(3) = 04\ 00\ 00\ 00$ $y_3 \oplus Rcon(3) = 12\ 66\ b4\ 8e = z_3$
$w_{12} = w_8 \oplus z_3 = c0\ af\ df\ 39$ $w_{13} = w_{12} \oplus w_9 = 89\ 2f\ 6b\ 67$ $w_{14} = w_{13} \oplus w_{10} = 57\ 51\ ad\ 06$ $w_{15} = w_{14} \oplus w_{11} = b1\ ae\ 7e\ c0$	$RotWord(w_{15}) = ae\ 7e\ c0\ b1 = x_4$ $SubWord(x_4) = e4\ f3\ ba\ c8 = y_4$ $Rcon(4) = 08\ 00\ 00\ 00$ $y_4 \oplus Rcon(4) = ec\ f3\ ba\ c8 = 4$

Key Expansion

Key Words	Auxiliary Function
$w_{16} = w_{12} \oplus z_4 = 2c\ 5c\ 65\ f1$ $w_{17} = w_{16} \oplus w_{13} = a5\ 73\ 0e\ 96$ $w_{18} = w_{17} \oplus w_{14} = f2\ 22\ a3\ 90$ $w_{19} = w_{18} \oplus w_{15} = 43\ 8c\ dd\ 50$	$RotWord(w_{19}) = 8c\ dd\ 50\ 43 = x_5$ $SubWord(x_5) = 64\ c1\ 53\ 1a = y_5$ $Rcon(5) = 10\ 00\ 00\ 00$ $y_5 \oplus Rcon(5) = 74\ c1\ 53\ 1a = z_5$
$w_{20} = w_{16} \oplus z_5 = 58\ 9d\ 36\ eb$ $w_{21} = w_{20} \oplus w_{17} = fd\ ee\ 38\ 7d$ $w_{22} = w_{21} \oplus w_{18} = 0f\ cc\ 9b\ ed$ $w_{23} = w_{22} \oplus w_{19} = 4c\ 40\ 46\ bd$	$RotWord(w_{23}) = 40\ 46\ bd\ 4c = x_6$ $SubWord(x_6) = 09\ 5a\ 7a\ 29 = y_6$ $Rcon(6) = 20\ 00\ 00\ 00$ $y_6 \oplus Rcon(6) = 29\ 5a\ 7a\ 29 = z_6$
$w_{24} = w_{20} \oplus z_6 = 71\ c7\ 4c\ c2$ $w_{25} = w_{24} \oplus w_{21} = 8c\ 29\ 74\ bf$ $w_{26} = w_{25} \oplus w_{22} = 83\ e5\ ef\ 52$ $w_{27} = w_{26} \oplus w_{23} = cf\ a5\ a9\ ef$	$RotWord(w_{27}) = a5\ a9\ ef\ cf = x_7$ $SubWord(x_7) = 06\ d3\ bf\ 8a = y_7$ $Rcon(7) = 40\ 00\ 00\ 00$ $y_7 \oplus Rcon(7) = 46\ d3\ df\ 8a = z_7$
$w_{28} = w_{24} \oplus z_7 = 37\ 14\ 93\ 48$ $w_{29} = w_{28} \oplus w_{25} = bb\ 3d\ e7\ f7$ $w_{30} = w_{29} \oplus w_{26} = 38\ d8\ 08\ a5$ $w_{31} = w_{30} \oplus w_{27} = f7\ 7d\ a1\ 4a$	$RotWord(w_{31}) = 7d\ a1\ 4a\ f7 = x_8$ $SubWord(x_8) = ff\ 32\ d6\ 68 = y_8$ $Rcon(8) = 80\ 00\ 00\ 00$ $y_8 \oplus Rcon(8) = 7f\ 32\ d6\ 68 = z_8$

Key Expansion

$w_{32} = w_{28} \oplus z_8 = 48\ 26\ 45\ 20$ $w_{33} = w_{32} \oplus w_{29} = f3\ 1b\ a2\ d7$ $w_{34} = w_{33} \oplus w_{30} = cb\ c3\ aa\ 72$ $w_{35} = w_{34} \oplus w_{32} = 3c\ be\ 0b\ 3$	$RotWord(w_{35}) = be\ 0b\ 38\ 3c = x_9$ $SubWord(x_9) = ae\ 2b\ 07\ eb = y_9$ $Rcon(9) = 1B\ 00\ 00\ 00$ $y_9 \oplus Rcon(9) = b5\ 2b\ 07\ eb = z_9$
$w_{36} = w_{32} \oplus z_9 = fd\ 0d\ 42\ cb$ $w_{37} = w_{36} \oplus w_{33} = 0e\ 16\ e0\ 1c$ $w_{38} = w_{37} \oplus w_{34} = c5\ d5\ 4a\ 6e$ $w_{39} = w_{38} \oplus w_{35} = f9\ 6b\ 41\ 56$	$RotWord(w_{39}) = 6b\ 41\ 56\ f9 = x_{10}$ $SubWord(x_{10}) = 7f\ 83\ b1\ 99 = y_{10}$ $Rcon(10) = 36\ 00\ 00\ 00$ $y_{10} \oplus Rcon(10) = 49\ 83\ b1\ 99 = z_{10}$
$w_{40} = w_{36} \oplus z_{10} = b4\ 8e\ f3\ 52$ $w_{41} = w_{40} \oplus w_{37} = ba\ 98\ 13\ 4e$ $w_{42} = w_{41} \oplus w_{38} = 7f\ 4d\ 59\ 20$ $w_{43} = w_{42} \oplus w_{39} = 86\ 26\ 18\ 76$	

Initial Transformation

- This step is also known as add round key.
- Block size of 128 bits are created from plaintext.
- $P = (p_0, p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8, p_9, p_{10}, p_{11}, p_{12}, p_{13}, p_{14}, p_{15})$
- $K_0 = (k_0, k_1, k_2, k_3, k_4, k_5, k_6, k_7, k_8, k_9, k_{10}, k_{11}, k_{12}, k_{13}, k_{14}, k_{15})$
- $P + K_0 = p_0 + k_0, p_1 + k_1, \dots, p_{15} + k_{15}$
- 0123456789abcdeffedcba9876543210 (plaintext)

P					K ₀					Result			
01	89	fe	76	+	0f	47	0c	af	=	0e	ce	f2	d9
23	ab	dc	54		15	d9	b7	7f		36	72	6b	2b
45	cd	ba	32		71	e8	ad	67		34	25	17	55
67	ef	98	10		c9	59	d6	98		ae	b6	4e	88

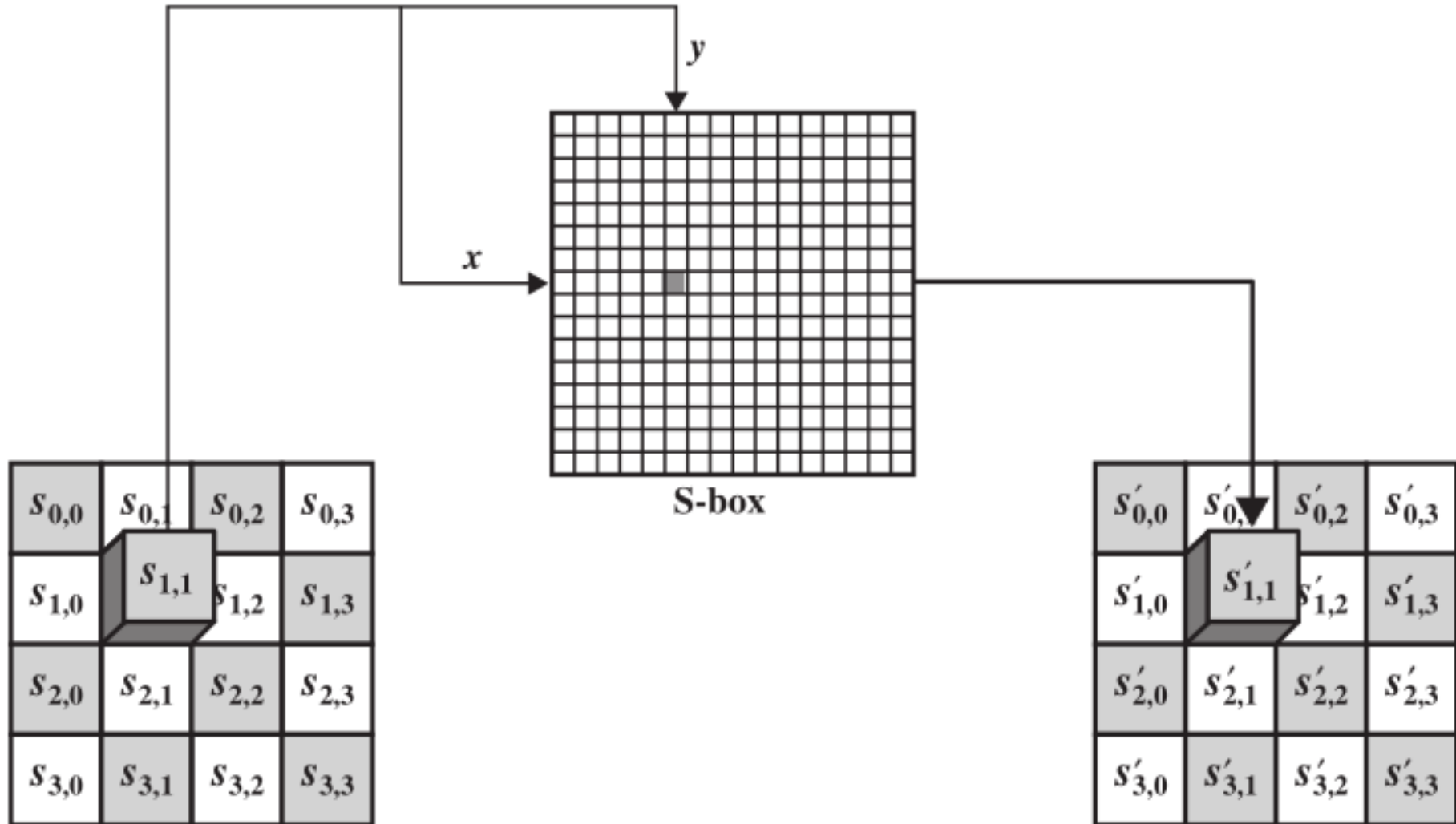
Round Transformation

- AES can be implemented in 10, 12 or 14 rounds.
- Each round consists of following transformations for generating strong ciphers.
- SubBytes
- ShiftRows
- MixColumns
- AddRoundKey
- Last round does not have mix column transformation.

Round Transformation

- **SubBytes**
- This operations is also called substitute byte transformation.
- The each byte in the state is mapped to new byte from the S-box.
- AES defines (16,16) S-box matrix of bytes.
- The first value in state byte is x values and second is y value.
- Now corresponding byte in S-box located at x row and y column is substituted.

Round Transformation: SubBytes



Round Transformation: SubBytes

- The state after initial transformation is

0e	ce	f2	d9
36	72	6b	2b
34	25	17	55
ae	b6	4e	88

- The first byte in this state is 0e
- It means row is 0 and column is e
- Now the byte is 0 row and e column in S-box is ab.
- So, 0e is mapped to ab.
- Similarly, we can map other 15 bytes.

Round Transformation: SubBytes

After initial
transformation

0e	ce	f2	d9
36	72	6b	2b
34	25	17	55
ae	b6	4e	88



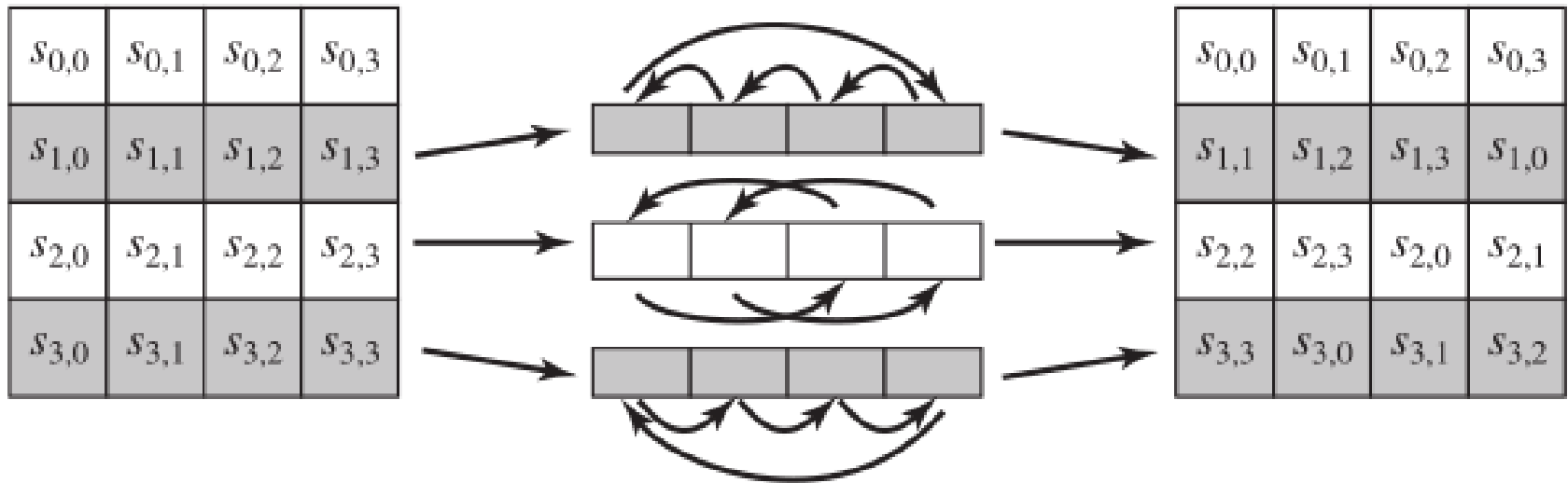
After SubBytes

ab	8b	89	35
05	40	7f	f1
18	3f	f0	fc
e4	4e	2f	c4

Round Transformation: ShiftRows

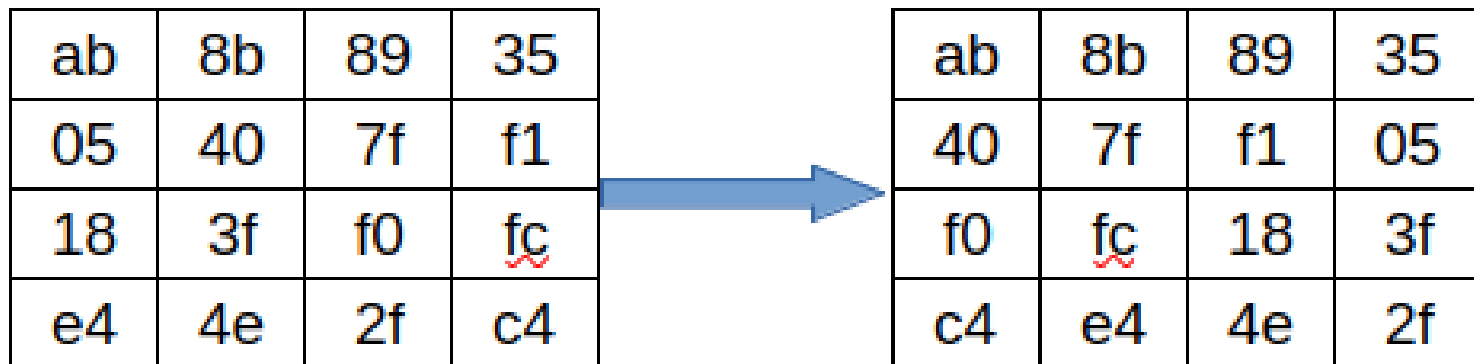
- Note that for convenience, the state is arranged in (4,4) byte matrix.
- The first column is first word, second column is second word and so on.
- In ShiftRow transformation, first row of the state is not altered.
- Second row is circular left shifted by 1 byte.
- Third row is circular left shifted by 2 bytes.
- Fourth row is left shifted by 3 bytes.

Round Transformation: ShiftRows



After SubBytes

After ShiftRows



Round Transformation MixColumn

- At this stage, the state matrix consists of the result of RowShift.
- Each byte of a column in state matrix is mapped into a new value that is a function of all four bytes in that column.
- The transformation can be defined by the following matrix multiplication on State

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Round Transformation MixColumn

- First matrix is constant matrix and second is state matrix.
- Note that for convenience, the state is arranged in (4,4) byte matrix.
- The first column is first word, second column is second word and so on.
- Constant matrix is designed in such a way that the each column of state matrix is transformed to a new value that is a function of all four bytes in that column.
- Here matrix multiplication is a field operation in $GF(2^8)$ with prime polynomial $x^8 + x^4 + x^3 + x + 1$.⁴⁵

Round Transformation MixColumn

- Multiplication of bytes is done in $GF(2^8)$ with modulus $x^8 + x^4 + x^3 + x + 1$.
- Addition is the same as XORing of 8-bit words.
- So, the numbers in constant matrix are polynomials.
- It means 03 in constant matrix is 00000011 which is $x+1$ and so on.
- After matrix multiplication, we get following equations

Round Transformation MixColumn

$$s'_{0,j} = (2 \cdot s_{0,j}) \oplus (3 \cdot s_{1,j}) \oplus s_{2,j} \oplus s_{3,j}$$

$$s'_{1,j} = s_{0,j} \oplus (2 \cdot s_{1,j}) \oplus (3 \cdot s_{2,j}) \oplus s_{3,j}$$

$$s'_{2,j} = s_{0,j} \oplus s_{1,j} \oplus (2 \cdot s_{2,j}) \oplus (3 \cdot s_{3,j})$$

$$s'_{3,j} = (3 \cdot s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \cdot s_{3,j})$$

- We can compute transformed columns using above equations.
- Note that we can perform the multiplication of polynomials as xor operations in $\text{GF}(2^8)$ field.

Round Transformation MixColumn

- It follows that multiplication by x (i.e., 00000010 or 2) can be implemented as a 1-bit left shift followed by a conditional bitwise XOR with (00011011), which represents $(x^4 + x^3 + x + 1)$

$$x \times f(x) = \begin{cases} (b_6b_5b_4b_3b_2b_1b_00) & \text{if } b_7 = 0 \\ (b_6b_5b_4b_3b_2b_1b_00) \oplus (00011011) & \text{if } b_7 = 1 \end{cases}$$

- So, we need to convert $f(x)$ into binary representation.
- If its b_7 is 0, the left shift.
- Otherwise, left shift and XOR with 00011011.

Round Transformation MixColumn

- Let us see example.

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

ab	8b	89	35
40	7f	f1	05
f0	<u>fc</u>	18	3f
c4	e4	4e	2f

- We get following first column after matrix multiplication.
- $S'_{0,0} = 2 s_{0,0} + 3 s_{1,0} + s_{2,0} + s_{3,0}$
- $= 2. ab + 3. 40 + f0 + c4$
- $2. 10101011 + 3. 01000000 + 11110000 + 11000100$

Round Transformation MixColumn

- Let us see compute.
- 2. $10101011 \quad 01010110$
- $= 01010110 + 00011011 \quad 00011011$
- $= 01001101 \quad 01001101$
- Let us compute
- 3. $01000000 = 00000011 \times 01000000$
- $= 01000000 \times [00000001 + 00000010]$
- $= 01000000 \times 00000001 + 01000000 \times 00000010$
- $= 01000000 + 10000000 = 11000000$

Round Transformation MixColumn

- Now, let come back to
- 2. 10101011 + 3. 01000000 + 11110000 + 11000100
- = 01001101 + 11000000 + 11110000 + 11000100
- = 10111001
- = b9

01001101

11000000

11110000

11000100

10111001

Round Transformation MixColumn

- After matrix multiplication

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

After RowShift

ab	8b	89	35
40	7f	f1	05
f0	fc	18	3f
c4	e4	4e	2f



After MixColumn

b9	94	57	75
e4	8e	16	51
47	20	9a	3f
c5	d6	f5	3b

Round Transformation

AddRoundKey

- In this transformation, the 128 bits of State are bitwise XORed with the 128 bits of the round key.
- $B9 + dc = 10111001 + 11011100 = 01100101 = 65$

After MixColumn

K_1

10111001

11011100

01100101

b9	94	57	75
e4	8e	16	51
47	20	9a	3f
c5	d6	f5	3b

+

dc	9b	97	38
90	49	<u>fe</u>	81
37	<u>df</u>	72	15
b0	e9	3f	a7



65	0f	c0	4d
74	cf	e8	d0
70	ff	e8	2a
75	3f	ca	9c

AES Encryption Example

Plaintext:	0123456789abcdef fedcba9876543210
Key:	0f1571c947d9e8590cb7add6af7f6798
Ciphertext:	ff0b844a0853bf7c6934ab4364148fb9

Start of Round	After SubBytes	After ShiftRows	After MixColumns	Round Key
01 89 fe 76 23 ab dc 54 45 cd ba 32 67 ef 98 10				0f 47 0c af 15 d9 b7 7f 71 e8 ad 67 c9 59 d6 98
0e ce f2 d9 36 72 6b 2b 34 25 17 55 ae b6 4e 88	ab 8b 89 35 05 40 7f f1 18 3f f0 fc e4 4e 2f c4	ab 8b 89 35 40 7f f1 05 f0 fc 18 3f c4 e4 4e 2f	b9 94 57 75 e4 8e 16 51 47 20 9a 3f c5 d6 f5 3b	dc 9b 97 38 90 49 fe 81 37 df 72 15 b0 e9 3f a7
65 0f c0 4d 74 c7 e8 d0 70 ff e8 2a 75 3f ca 9c	4d 76 ba e3 92 c6 9b 70 51 16 9b e5 9d 75 74 de	4d 76 ba e3 c6 9b 70 92 9b e5 51 16 de 9d 75 74	8e 22 db 12 b2 f2 dc 92 df 80 f7 c1 2d c5 1e 52	d2 49 de e6 c9 80 7e ff 6b b4 c6 d3 b7 5e 61 c6
5c 6b 05 f4 7b 72 a2 6d b4 34 31 12 9a 9b 7f 94	4a 7f 6b bf 21 40 3a 3c 8d 18 c7 c9 b8 14 d2 22	4a 7f 6b bf 40 3a 3c 21 c7 c9 8d 18 22 b8 14 d2	b1 c1 0b cc ba f3 8b 07 f9 1f 6a c3 1d 19 24 5c	c0 89 57 b1 af 2f 51 ae df 6b ad 7e 39 67 06 c0

AES Encryption Example

71 48 5c 7d 15 dc da a9 26 74 c7 bd 24 7e 22 9c	a3 52 4a ff 59 86 57 d3 f7 92 c6 7a 36 f3 93 de	a3 52 4a ff 86 57 d3 59 c6 7a f7 92 de 36 f3 93	d4 11 fe 0f 3b 44 06 73 cb ab 62 37 19 b7 07 ec	2c a5 f2 43 5c 73 22 8c 65 0e a3 dd f1 96 90 50
f8 b4 0c 4c 67 37 24 ff ae a5 c1 ea e8 21 97 bc	41 8d fe 29 85 9a 36 16 e4 06 78 87 9b fd 88 65	41 8d fe 29 9a 36 16 85 78 87 e4 06 65 9b fd 88	2a 47 c4 48 83 e8 18 ba 84 18 27 23 eb 10 0a f3	58 fd 0f 4c 9d ee cc 40 36 38 9b 46 eb 7d ed bd
72 ba cb 04 1e 06 d4 fa b2 20 bc 65 00 6d e7 4e	40 f4 1f f2 72 6f 48 2d 37 b7 65 4d 63 3c 94 2f	40 f4 1f f2 6f 48 2d 72 65 4d 37 b7 2f 63 3c 94	7b 05 42 4a 1e d0 20 40 94 83 18 52 94 c4 43 fb	71 8c 83 cf c7 29 e5 a5 4c 74 ef a9 c2 bf 52 ef
0a 89 c1 85 d9 f9 c5 e5 d8 f7 f7 fb 56 7b 11 14	67 a7 78 97 35 99 a6 d9 61 68 68 0f b1 21 82 fa	67 a7 78 97 99 a6 d9 35 68 0f 61 68 fa b1 21 82	ec 1a c0 80 0c 50 53 c7 3b d7 00 ef b7 22 72 e0	37 bb 38 f7 14 3d d8 7d 93 e7 08 a1 48 f7 a5 4a

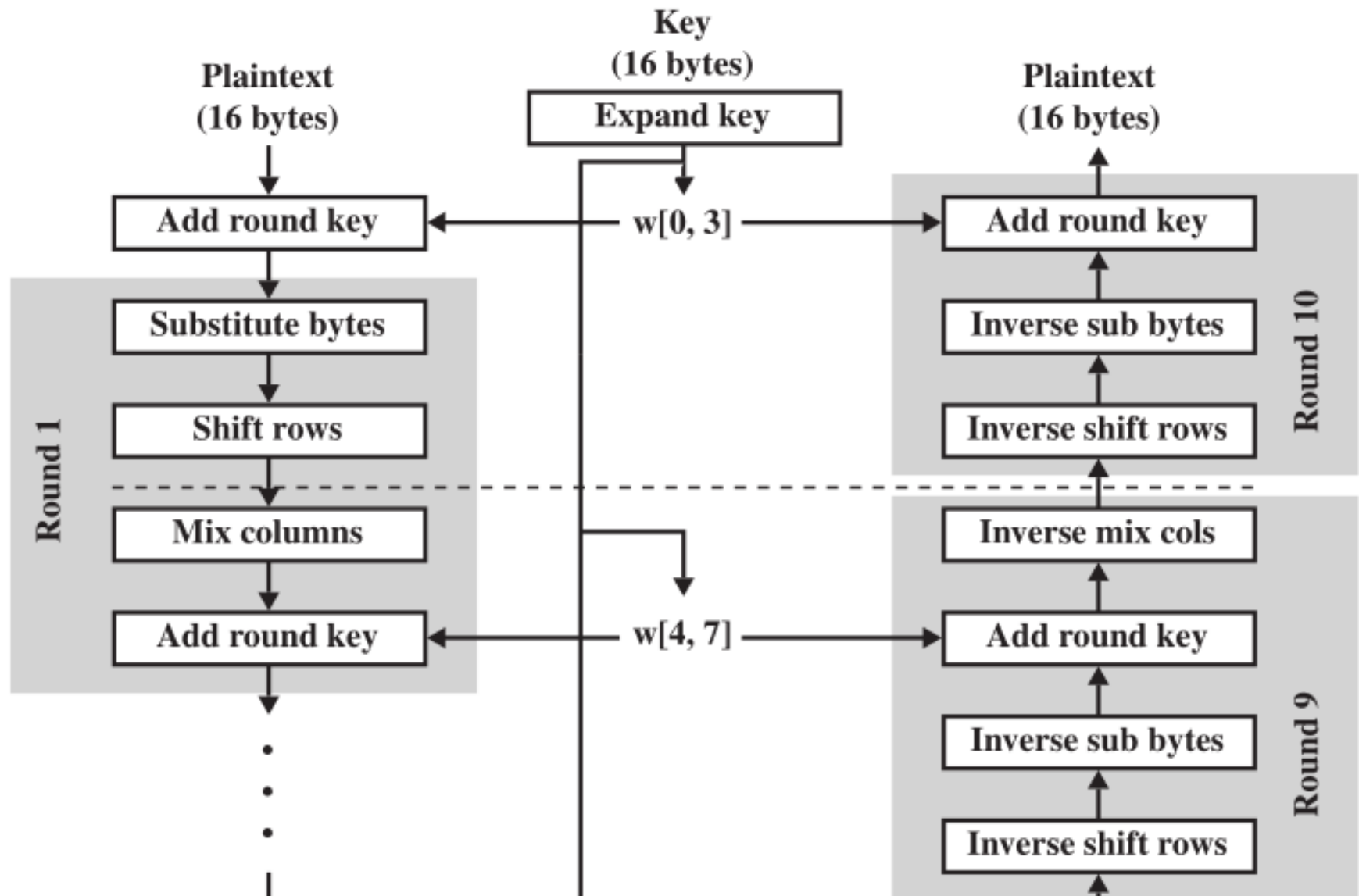
AES Encryption Example

db a1 f8 77 18 6d 8b ba a8 30 08 4e ff d5 d7 aa	b9 32 41 f5 ad 3c 3d f4 c2 04 30 2f 16 03 0e ac	b9 32 41 f5 3c 3d f4 ad 30 2f c2 04 ac 16 03 0e	b1 1a 44 17 3d 2f ec b6 0a 6b 2f 42 9f 68 f3 b1	48 f3 cb 3c 26 1b c3 be 45 a2 aa 0b 20 d7 72 38
f9 e9 8f 2b 1b 34 2f 08 4f c9 85 49 bf bf 81 89	99 1e 73 f1 af 18 15 30 84 dd 97 3b 08 08 0c a7	99 1e 73 f1 18 15 30 af 97 3b 84 dd a7 08 08 0c	31 30 3a c2 ac 71 8c c4 46 65 48 eb 6a 1c 31 62	fd 0e c5 f9 0d 16 d5 6b 42 e0 4a 41 cb 1c 6e 56
cc 3e ff 3b a1 67 59 af 04 85 02 aa a1 00 5f 34	4b b2 16 e2 32 85 cb 79 f2 97 77 ac 32 63 cf 18	4b b2 16 e2 85 cb 79 32 77 ac f2 97 18 32 63 cf	4b 86 8a 36 b1 cb 27 5a fb f2 f2 af cc 5a 5b cf	b4 ba 7f 86 8e 98 4d 26 f3 13 59 18 52 4e 20 76
ff 08 69 64 0b 53 34 14 84 bf ab 8f 4a 7c 43 b9				

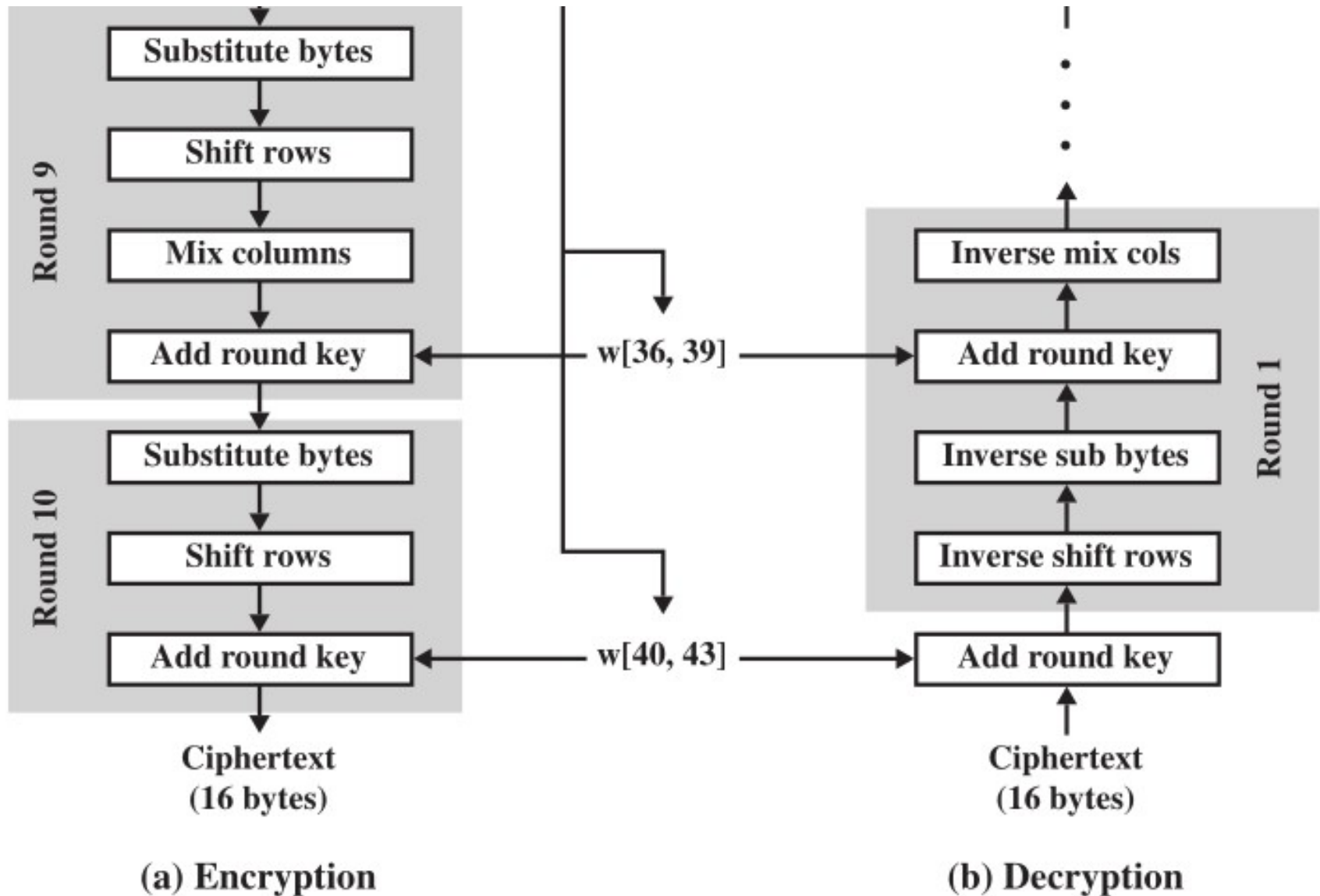
AES Decryption

- Add K_{10} round key with ciphertext
- Perform inverse round transformation
- After last round transformation, we will get plaintext.

AES Decryption



AES Decryption



AES Decryption: AddRoundKey

- AddRoundKey is same to AddRoundKey to that of Encryption.
- But in this stage, last round key (K_{10}) is XORed with cipher text.
- The result is stored in state matrix.

AES Decryption: Round Transformation

- In round transformation, following transformations are performed one after another.
- Inverse shift rows
- Inverse sub bytes
- Add round key
- Inverse mix cols
- The last round does not have Inverse mid cols transformation.

AES Decryption: InverseShiftRows

- InvShiftRows performs the circular right shifts in the for each of the last three rows
- Second row of state matrix is circular right shift with 1-byte
- Third row with 2 bytes.
- Fourth row with 3 bytes.

AES Decryption: InverseSubBytes

- InvSubBytes makes use of the inverse S-box for substituting bytes of state matrix.
- For substitution, the location of byte in inverse S-box is found using the byte to be substituted.
- Byte 2A means second row and A column in inverse S-box.
- At this location, byte 95 is present.
- So, 2A is substituted with 95.
- Inverse S-box is given below.

AES Decryption: Inverse Shift Bytes

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(b) Inverse S-box

AES Decryption: InverseSubBytes

- InvSubBytes makes use of the inverse S-box for substituting bytes of state matrix.
- For substitution, the location of byte in inverse S-box is found using the byte to be substituted.
- Byte 2A means second row and A column in inverse S-box.
- At this location, byte 95 is present.
- So, 2A is substituted with 95.
- Inverse S-box is given below.

AES Decryption: AddRoundKey

- Inverse add round key is same to decryption add round key.
- It is because xor operation is its own inverse.

AES Decryption: InverseMixCols

- InvMixColumns is defined by the following matrix multiplication:

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

AES Properties

- **Avalanche Effect:**
- A small change in plaintext or key produces a large change in the ciphertext.

Round		Number of Bits that Differ
	0123456789abcdef fedcba9876543210 0023456789abcdef fedcba9876543210	1
0	0e3634aece7225b6f26b174ed92b5588 0f3634aece7225b6f26b174ed92b5588	1
10	ff0b844a0853bf7c6934ab4364148fb9 612b89398d0600cde116227ce72433f0	58